

# Multilabel-классификатор изображений для отбора VLM-данных

Прикладная бизнес-задача с работы для Junior ML Contest 2026

Леонтенков Егор Евгеньевич

Москва, Россия

egorleontenkov@yandex.ru

**Аннотация**—Проект направлен на разработку multilabel-классификатора изображений для анализа датасета, а также отбора и фильтрации данных в пайплайнах обучения Vision-Language моделей. В современных подходах качество обучающих данных напрямую влияет на способность модели решать прикладные задачи: понимать документы, таблицы, рукописный текст и другие домены. Однако открытые источники данных и пользовательские изображения из GigaChat содержат огромное количество различных изображений, включая нерелевантные, зашумлённые и низкокачественные примеры. Полная разметка всех изображений является слишком дорогой и медленной, поэтому возникает потребность в дешёвом первичном фильтре, который способен быстро выделять нужные домены и отсекал нерелевантные данные.

В рамках проекта был разработан полный ML-пайплайн: классификация визуальных доменов, генерация кандидатов на разметку через CLIP, авторазметка изображений, отбор данных для активного обучения, обучение multilabel-модели, сравнение архитектур и инференс в YTSaurus. Полученный классификатор используется как инфраструктурная компонента для дешёвой предварительной классификации изображений в датасетах, отбора нужных срезов данных и снижения стоимости последующих VLM-этапов. Проект был выполнен в рамках работы в команде GigaChat VLM Pretrain. Код доступен по ссылке: <https://github.com/Vsemir13/jmlc>.

**Index Terms**—computer vision, multilabel classification, dataset, vision-language models, active learning, CLIP, Dawid-Skene, PyTorch, MLOps, YTSaurus

## I. Задача и продуктовая значимость

Современные VLM обучаются на больших и разнородных наборах изображений, поэтому качество модели зависит не только от объёма данных, но и от распределений в датасете. Без автоматического анализа большие потоки данных превращаются в неизвестный набор изображений, где сложно контролировать баланс доменов, находить шумные срезы и собирать выборки для последующей разметки.

Современные источники данных содержат много потенциально полезных изображений, но также включают размытые картинки, пустые изображения, скриншоты без полезного содержания и другие нерелевантные примеры. Ключевая продуктовая проблема заключается в том, что полная обработка всех изображений ассессорами или VLM-моделями слишком затратна: она требует дорогого GPU-инференса, длительного времени обработки и ручной поддержки инфраструктуры. Поэтому в пайплайне подготовки данных нужен более дешёвый подход - классификатор,

который быстро определяет домены изображения и отсекает мусор.

### Масштаб продуктовой задачи:

- 600 тыс.+ запросов пользователей в день;
- 2 млрд+ изображений в датасете;
- 38% изображений в контрольной выборке оказались нерелевантными для обучения VLM;
- Нет автоматической фильтрации данных;
- Неизвестно содержание обучающего датасета.

Эти метрики подтверждают, что ежедневный поток данных слишком велик для ручной или полной VLM-обработки, поэтому пайплайн подготовки датасета требует автоматизированной фильтрации и контроля распределения данных.

Цель проекта — разработать multilabel-классификатор, который позволяет находить низкокачественные и нерелевантные изображения, покрывает широкий набор визуальных доменов, работает достаточно быстро даже на CPU и интегрируется в существующие пайплайны обработки данных.

Особенность задачи состоит в том, что классы описывают разные аспекты изображения: домен, формат, содержательные элементы, артефакты и качество. Поэтому одно изображение может одновременно содержать код, таблицу, интерфейсные элементы и водяные знаки, а модель должна не выбирать один класс, а независимо предсказывать набор применимых меток.

Мой личный вклад включал полный цикл разработки решения: формирование классов, построение кандидатов для обучения через CLIP, запуски VLM-разметок, реализацию активного обучения, обучение, сравнение моделей, анализ метрик и реализацию инференса через YT MapReduce.

## II. Предметная область

Первым этапом проекта стало проектирование таксономии визуальных доменов и правил разметки. Полный список классов не раскрывается из-за ограничений NDA, однако итоговая схема включала 41 класс и описывала не только предметные домены, но и мета-признаки изображения: качество, водяные знаки, пустые изображения, шум и другие артефакты. Таксономия была организована как многоэтапный пайплайн принятия решений: сначала

проверяются признаки качества, затем водяные знаки, после чего выбираются доменные метки по строгим визуальным признакам.

- полезные VLM-домены: математика, таблицы, графики, код, документы, интерфейсы;
- специализированные сценарии: карты, технические чертежи, научные и медицинские исследования;
- мета и фильтрационные признаки: low quality, blurry, watermarked, junk.

### III. Общий подход к решению

Основная идея решения заключалась в том, чтобы использовать сильные VLM модели и CLIP эмбединги не как постоянный дорогой инференс на всех изображениях, а как источник разметки и отбора кандидатов для обучения более дешёвого классификатора.

#### Общая схема решения:

Images → CLIP candidate mining → VLM  
autolabeling → active learning sampling → train  
dataset → multilabel classifier → YTSaurus  
inference

Входом в пайплайн являются большие таблицы с изображениями и таксономия классов. Для изображений и текстовых описаний классов считаются эмбединги, а после формируются кандидаты на каждый класс. Эти кандидаты размечаются несколькими VLM с подробным промптом, а ответы агрегируются в итоговые multilabel-метки, после чего на данных обучается классификатор. На следующих итерациях используется active learning: классификатор прогоняется по большим неразмеченным данным и отбираются новые примеры, которые отправляются на доразметку.

Такой подход позволяет одновременно решить три задачи: обеспечить хорошее распределение классов, снизить стоимость разметки и получить модель, которую можно дообучать и применять в production-пайплайне.

### IV. Данные и пайплайн разметки

Данные поступали из нескольких источников: Common Crawl изображения из открытого интернета, пользовательских запросов GigaChat Vision и небольших доменных наборов, подготовленных аналитиками для отдельных визуальных категорий. Эти источники различались по качеству и распределению доменов, поэтому перед обучением требовался отбор кандидатов, авторазметка и последующая агрегация меток.

#### A. Поиск кандидатов через CLIP

Для изображений были посчитаны эмбединги с помощью CLIP-модели PE-Core-L14-336. Также были посчитаны текстовые эмбединги описаний классов. После этого оценивалось расстояние между эмбедингом изображения и эмбедингами описаний классов. По ближайшим классам выбирались кандидаты на разметку.

Такой подход решил проблему покрытия классов, так как случайное семплирование изображений из источников данных могло пропустить редкие домены. Майнинг кандидатов через CLIP позволил заранее собрать более релевантные примеры для каждого класса и повысить вероятность того, что все категории будут представлены в обучении. На данном этапе датасет содержал около 2 млн изображений.

#### B. VLM-разметка

Для разметки кандидатов использовались три сильные VLM: Qwen2.5-VL-72B-Instruct, GLM-4.6V, Qwen3-VL-235B-A22B-Instruct. Каждая модель получала изображение и промпт с описанием таксономии, после чего возвращала JSON с набором классов. Использование нескольких моделей было направлено на учёт сильных сторон разных VLM и снижение смещений отдельных разметчиков.

Далее ответы моделей агрегировались методом Dawid-Skene и в результате для каждого изображения формировалась строка в таблице с бинарными соответствиями классу. Такой подход позволил учитывать различия в качестве разметчиков и получить более надежную разметку.

#### C. Active learning

После первых версий модели было реализовано активное обучение: классификатор запускался на неразмеченных данных, вероятности анализировались по классам и уровням уверенности, а в доразметку попадали примеры из разных групп уверенности в предсказании. Это позволяло эффективнее использовать бюджет разметки, чем случайное семплирование.

### V. Инженерия и методы обучения

Задача была сформулирована как multilabel image classification. Для экспериментов были выбраны архитектуры с разным балансом качества и стоимости инференса: EVA02-large и EVA02-M как сильные современные vision-transformer-ы, EfficientNetV2-M как более лёгкая модель с хорошим соотношением качества и скорости, а CSAT-v2 как дополнительный baseline для сравнения.

Для борьбы с дисбалансом классов использовался Distribution-Balanced Loss, потому что распределение доменов в данных разнообразно и одни классы встречаются часто, а другие сильно реже. Без учёта дисбаланса модель могла бы переобучиться на популярные домены и хуже распознавать редкие категории.

Проект был реализован как модульная ML-система с воспроизводимым окружением и разделением основных этапов пайплайна: подготовка данных, авторазметка, фильтрация, обучение, оценка качества и инференс. Такая архитектура позволяла масштабировать эксперименты, сравнивать разные версии датасетов и моделей.

Разработка велась в GitLab с использованием Git, Docker, линтеров качества кода, pytest-тестов, YAML-конфигураций и логирования экспериментов. Docker-образы использовались для воспроизводимого запуска разметки, обучения и инференса в разных окружениях, а

YAML-конфигурации фиксировали гиперпараметры моделей.

Для конкурсной подачи был подготовлен публичный GitHub-репозиторий с воспроизводимой и обезличенной версией проекта: внутренние пути и сервисные детали компании были удалены, но сохранены основные компоненты ML-пайплайна, демо-данные и скрипты запуска. Подробности реализации, структура кода и инструкции по запуску приведены в README репозитория.

Данные и артефакты были разделены между S3 и YTSaurus: в S3 хранились изображения и checkpoint-ы моделей, а большие таблицы обрабатывались в YTSaurus через распределённые job-ы. Авторазметка запускалась через Apache Airflow: для каждой VLM-модели-разметчика использовалось отдельное Docker-окружение с зафиксированными зависимостями, параметрами запуска и версиями промптов.

Обучение моделей проводилось на GPU-кластерах Cloud.ru, а метрики и параметры запусков логировались в Weights & Biases для сравнения версий моделей и выбора checkpoint-ов. Также был реализован инференс классификатора в YTSaurus как MapReduce job, где входная таблица содержала base64-изображения, reducer загружал код, checkpoint и config, выполнял preprocessing и сохранял в выходную таблицу вектор предсказаний по классам. Такой CPU-инференс позволял масштабировать фильтрацию на большие таблицы, не занимая GPU-кластеры, необходимые для обучения и экспериментов.

## VI. Эксперименты и результаты

Экспериментальная часть включала сравнение нескольких архитектур и checkpoint-ов. Качество оценивалось на отложенной размеченной выборке с тем же набором классов, а для сравнения моделей использовались агрегированные multilabel-метрики precision, recall, ROC-AUC, PR-AUC и F1-score.

Таблица I  
Сравнение моделей

| Модель           | Params | Prec.        | Rec.         | ROC          | PR           | F1           |
|------------------|--------|--------------|--------------|--------------|--------------|--------------|
| EVA02-large      | 304M   | <b>0.894</b> | <b>0.849</b> | <b>0.997</b> | <b>0.954</b> | <b>0.870</b> |
| EVA02-M          | 86M    | 0.881        | 0.808        | 0.996        | 0.936        | 0.842        |
| EfficientNetV2-M | 54M    | 0.866        | 0.750        | 0.993        | 0.912        | 0.803        |
| CSAT-v2          | 21M    | 0.855        | 0.657        | 0.990        | 0.872        | 0.740        |

Из результатов видно, что семейство EVA02 стало лучшим среди сравниваемых архитектур: обе модели заметно опережают EfficientNetV2-M и CSAT-v2 по ключевым метрикам. EVA02-large показала максимальное качество, однако её размер составляет 304M параметров. Для итогового решения была выбрана EVA02-M, так как при 86M параметров она сохраняет близкое качество и даёт лучший баланс между точностью и стоимостью инференса.

## VII. Продуктовая польза

Практическая ценность проекта заключается в том, что классификатор был интегрирован как production-инструмент для управления vision данными и используется командой в регулярном пайплайне подготовки датасетов. На масштабе 2 млрд+ изображений он позволяет анализировать распределения по классам, находить шумные срезы, отбирать изображения нужных категорий и формировать выборки для последующих разметок.

В качестве альтернатив рассматривались случайное семплирование, полная VLM-разметка и ручная проверка ассессорами, однако эти подходы либо плохо покрывали редкие домены, либо плохо масштабировались на миллиарды изображений.

Для оценки порядка экономического эффекта можно использовать открытые тарифы Yandex Cloud DataSphere, где A100 стоит 542 руб/час, а CPU-конфигурация с Icelake и 32 vCPU - 150 руб/час. При скорости VLM-разметки 0.1 секунды на изображение и худшем измеренном CPU-инференсе классификатора 16 изображений за 3 секунды полный инференс на 2 млрд изображений стоил бы порядка 30 млн руб. для VLM и 15.6 млн руб. для специализированного классификатора, то есть примерно в 1.9 раза дешевле.

Таким образом, результат проекта не ограничился локальным улучшением метрик, а стал переиспользуемым инструментом команды для контроля состава датасетов и подготовки данных под разные VLM-сценарии.

### Приложение А

#### Использование AI-инструментов в работе

В работе над проектом использовались AI-инструменты как вспомогательная часть разработки, анализа и разметки данных. На уровне данных сильные Vision-Language модели Qwen2.5-VL-72B-Instruct-AWQ, GLM-4.6V и Qwen3-VL-235B-A22B-Instruct-AWQ применялись для авторазметки изображений.

AI-инструменты применялись для написания и рефакторинга вспомогательного кода, подготовки pytest-тестов, помощи с Docker-окружением, YAML-конфигурациями и Airflow DAG, а также для исследования подходов к multilabel-классификации, active learning и работе с дисбалансом классов. В работе с данными агент использовался для итеративного улучшения промптов VLM-авторазметки, уточнения классов и анализа типовых ошибок разметчиков.

Использование AI-инструментов было ограничено безопасным контуром: секреты, токены, внутренние endpoints, production-пути и чувствительные данные не передавались внешним инструментам. Все ключевые решения по постановке задачи, таксономии, пайплайну разметки, выбору моделей, экспериментам и анализу результатов принимались мной.